Informe del proyecto Pulso

Implementación de SDD con GitHub Spec Kit

Trabajo grupal de cuatro integrantes
29 de octubre de 2026
Producto 1.0.0 | Informe ampliado de proceso y validación

Resumen ejecutivo

Como equipo desarrollamos Pulso, un rastreador de criptomonedas que consulta CoinGecko y presenta precios en USD, variación y volumen de 24 horas. La página permite buscar por nombre o símbolo, guardar favoritos y explorar gráficos de 24 horas, 7, 30 y 90 días. El propósito del trabajo es demostrar cómo una especificación orienta la implementación y permite verificar sus resultados.

Este informe reúne el proceso seguido, la discusión del código generado y una revisión organizada según las fases de Piskala (2026). El artículo sitúa la revisión humana entre especificación, planificación e implementación, y combina pruebas y juicio humano en la validación [1, pp. 3-4]. Su descripción de Kiro se usa como marco comparativo; la herramienta utilizada en Pulso fue GitHub Spec Kit.

La validación técnica fue realizada con asistencia de IA y herramientas ejecutables. Se volvieron a ejecutar **20 pruebas automatizadas y todas aprobaron**, además de comprobar la sintaxis de los módulos. La aprobación humana final del equipo queda por registrar en el acta de la sección 5. Los hallazgos técnicos de este informe no sustituyen esa decisión.

Alcance de la entrega

Incluido	Fuera del alcance
Criptomonedas, cotizaciones USD y cuatro rangos históricos.	Cotizaciones bursátiles como AAPL y ejecución de operaciones.
Búsqueda, favoritos locales, ordenación y gráficos de precio o volumen.	Cuentas propias, portafolios con saldos y streaming de precios.
Especificación, plan, tareas, contratos, pruebas y evidencias.	Certificación de seguridad o auditoría integral de accesibilidad.

La aplicación está disponible en https://pulso-mercado-sdd.deepgiant66.chatgpt.site con acceso privado inicial del propietario. El informe web y el código acompañan la entrega. El enlace puede requerir acceso a la cuenta propietaria.

1 Fundamento del flujo SDD y relación con Kiro

La especificación como contrato

Piskala plantea que la especificación expresa la intención del sistema y que el código debe implementarla. Distingue spec-first, donde la especificación guía el desarrollo inicial; spec-anchored, donde se mantiene sincronizada con el código; y spec-as-source, donde el código se regenera y las personas editan la especificación [1, pp. 1-3].

Pulso corresponde a un desarrollo inicial **spec-first con prácticas de trazabilidad orientadas a spec-anchored**. Conservamos los requisitos, los relacionamos con archivos y pruebas y registramos limitaciones. No afirmamos spec-as-source: el código recibió ediciones directas y no existe regeneración determinista completa. Tampoco se implementó una integración continua que compruebe cada commit.

Qué dice el artículo sobre Kiro

La sección V C describe Kiro como una herramienta que guía requisitos, diseño y creación de tareas antes de generar código [1, p. 5]. Esa descripción es más breve que el flujo general del artículo. Por ello, vinculamos esas tres etapas con Specify, Plan e Implement, y añadimos Validate como cierre del ciclo general. No se utilizó Kiro ni se atribuyen al proyecto archivos o ejecuciones de ese producto.

Fase del artículo	Aplicación en Pulso	Control humano esperado
Specify Especificar	Definir comportamiento, historias y criterios en spec.md.	Confirmar alcance, reglas y casos de aceptación.
Plan Planificar	Arquitectura, modelo y contrato en plan.md y documentos asociados.	Revisar viabilidad, datos y restricciones.
Implement Implementar	Descomponer en tareas y construir archivos revisables.	Contrastar código con requisitos y diseño.
Validate Validar	Pruebas, revisión de interfaz y evidencia de publicación.	Aceptar resultados o solicitar correcciones.

Aplicación proporcionada al proyecto

Adoptamos el nivel de detalle necesario para resolver ambigüedades reales: diferencia entre acciones y criptomonedas, moneda USD, rangos, volumen móvil, favoritos y fallos del proveedor. El artículo advierte sobre especificaciones burocráticas, desactualización y falsa confianza en pruebas aprobadas [1, p. 7]. En este caso, una prueba aprobada demuestra el comportamiento cubierto por esa prueba; no acredita por sí sola que el requisito represente toda la necesidad del grupo.

2 Proceso seguido para especificar y planificar

Interpretación del enunciado

Partimos de un rastreador que debía recibir un nombre o símbolo y mostrar precio, variación de 24 horas y volumen mediante CoinGecko. Se añadieron gráficos con distintas temporalidades. Interpretamos "criptomonedas / acciones" como alternativas y seleccionamos criptomonedas por coherencia con el proveedor elegido. Para AAPL y otros símbolos bursátiles habituales se muestra una explicación; no se reemplaza una acción por un token parecido.

Requisitos y criterios de aceptación

Se redactaron cuatro historias: consultar mercado, analizar histórico, mantener seguimiento personal y entregar documentación académica. Se asignaron FR-01 a FR-13 y SC-01 a SC-05 para poder rastrear comportamientos. Entre las reglas se incluyeron búsqueda sin distinguir mayúsculas, IDs de CoinGecko, datos ausentes como "—", cuatro rangos, persistencia de favoritos y tratamiento explícito de errores.

La distinción entre variación de 24 horas y rendimiento del período evita que la interfaz atribuya el mismo significado a porcentajes distintos. También se precisó que total_volume representa volumen móvil de 24 horas: los puntos del gráfico no deben sumarse como si fueran operaciones independientes de cada intervalo.

Instalación y procedencia de Spec Kit

Se instaló Specify CLI 1.2.5 en un entorno virtual de Python 3.12.14. La inicialización incorporó plantillas y scripts oficiales, integración Codex, modo de skills y PowerShell. El lanzador generado por pip fue bloqueado por Control de aplicaciones; se ejecutó el punto de entrada Python del mismo paquete sin modificar la política del equipo. Los logs de versión, inicialización y prerrequisitos se conservaron como evidencia.

Orden real y desviación metodológica

La especificación, investigación, plan y tareas iniciales se escribieron antes del código, mientras se instalaba el CLI. El scaffold formal terminó después de comenzar la implementación y entonces se completó la constitución con sus cinco principios. Por tanto, la ejecución no acredita revisiones humanas previas entre todas las fases ni reproduce el orden ideal de forma estricta. Esta diferencia se registra como aprendizaje del proceso.

Decisiones del plan

Elegimos HTML, CSS y módulos JavaScript sin dependencias de producción. El navegador consulta CoinGecko; core.js concentra API y cálculos, y app.js gestiona estado y eventos. Los favoritos viven en localStorage y el gráfico se representa en SVG. El servidor Node solo sirve archivos durante desarrollo. Se documentaron el modelo de datos, el contrato HTTP, la caché y los estados de carga, vacío, error y captura.

3 Proceso seguido para implementar y publicar

Construcción por responsabilidades

La implementación comenzó con el cliente API, los formatos numéricos y las funciones de normalización y estadísticas. Se construyeron después la estructura de la página, la tabla de mercado, la búsqueda y el detalle del activo. Los gráficos incorporaron rangos de 1, 7, 30 y 90 días, precio o volumen, ejes, tooltip y navegación con flechas. Finalmente se añadieron favoritos, filtros, orden y actualización manual.

La aplicación usa una caché de 60 segundos para mercado y de cinco minutos para históricos, deduplica consultas idénticas y aplica timeout de 12 segundos. Después de un 429 establece una pausa de 60 segundos; esta pausa reduce reintentos, pero no garantiza que el proveedor restablezca el acceso en ese momento. Las secuencias de solicitudes evitan aplicar respuestas obsoletas al activo o rango actual.

Datos de respaldo y fallos observados

Se obtuvo una captura real de 50 activos y cuatro históricos de Bitcoin a las 10:11 de Colombia del 29 de octubre de 2026. Contiene 288, 169, 721 y 2160 muestras para 24 horas, 7, 30 y 90 días, respectivamente. Son cantidades observadas en esa descarga, no valores garantizados para futuras consultas. Cuando falla el proveedor se identifica la captura con su fecha y se evita dibujar datos inventados.

Correcciones introducidas durante la validación

Se corrigió la recuperación de favoritos situados fuera del primer listado, consultando sus IDs cuando es posible. La revisión móvil detectó desbordamiento horizontal: el ancho desplazable era de 492 píxeles frente a 375 disponibles. Se posicionó el contenedor de la tabla y se limitaron nombres largos; después se verificó una relación de 375 a 375 píxeles. También se aplicó formato al código para facilitar su lectura.

Pruebas y publicación

Se ejecutaron comprobaciones de sintaxis y 20 pruebas de lógica y cliente HTTP. La revisión de navegador comprobó búsqueda, selección de Ethereum, temporalidades, volumen, favoritos, teclado, explicación de AAPL y modo de captura. El proceso de publicación empaquetó el estado exacto del código y Sites confirmó el despliegue. El historial conserva dos commits de preparación y cierre: 0830f2e y 8a177e8.

Artefacto	Función en el proceso
.specify/memory/constitution.md	Principios y restricciones del proyecto.
specs/001-crypto-tracker/	Especificación, investigación, plan, modelo, contrato y tareas.
dist/ y tests/core.test.mjs	Implementación ejecutable y 20 pruebas.
docs/evidence/	Logs, revisión de navegador, capturas y confirmación del despliegue.

4 Revisión guiada de requisitos y diseño

Organizamos la revisión en las cuatro fases del artículo. El análisis siguiente aporta criterios, observaciones y decisiones propuestas para el equipo. La inspección y las pruebas aquí documentadas fueron asistidas por IA; las personas del grupo deben confirmar la aceptación final.

Revisión de la especificación

Pregunta de revisión: ¿los requisitos describen el producto que necesitamos y permiten reconocer cuándo funciona correctamente? Según Piskala, Specify debe definir comportamiento y criterios verificables antes de imponer detalles de implementación [1, p. 3].

En Pulso encontramos un alcance comprobable: búsqueda, precio USD, cambio de 24 horas, volumen y gráficos por rango. La principal ambigüedad es la mención de acciones. Nuestra decisión propuesta es aceptar explícitamente criptomonedas para esta entrega; si el docente exige acciones bursátiles, habrá que ajustar la especificación y añadir un proveedor apropiado antes de programar esa ampliación.

También se revisan errores y significado de datos. Una captura antigua debe distinguirse de una lectura nueva, un dato ausente no debe convertirse en cero y el volumen móvil no debe describirse como volumen independiente del intervalo. Estas condiciones son parte de la corrección funcional, no solo del diseño visual.

Criterio de aceptación humana: el grupo confirma que los 13 FR cubren el enunciado acordado, que USD y los cuatro rangos son suficientes y que la exclusión de acciones está aceptada. Si cambia una regla, se actualizan spec.md y sus pruebas antes de modificar el comportamiento.

Revisión del plan y el diseño

Pregunta de revisión: ¿la arquitectura puede cumplir esos requisitos de forma comprensible y mantenible? Plan debe explicar componentes, interfaces, modelos y restricciones [1, p. 3]. La separación entre core.js y app.js facilita probar cálculos y fallos de red sin depender de la interfaz completa.

El sitio estático resulta proporcionado para una consulta académica sin cuentas ni transacciones. El almacenamiento local cubre favoritos; no justifica una base de datos. El acceso a CoinGecko sin clave funcionó durante la construcción, pero no constituye una garantía de disponibilidad. Si se adopta una clave, deberá manejarse en un servidor y no incorporarse al navegador.

Criterio de aceptación humana: el equipo puede explicar la ruta usuario → interfaz → cliente API → proveedor; reconoce que los favoritos dependen del navegador y acepta las limitaciones del respaldo. Antes de pasar a tareas, debe quedar registrada la decisión sobre autenticación y sobre datos actuales frente a captura.

5 Revisión guiada del código y la aceptación

Revisión de la implementación

Pregunta de revisión: ¿cada tarea construyó lo acordado sin introducir supuestos nuevos? Piskala propone tareas pequeñas y revisables, con comprobación contra especificación y plan [1, p. 4]. En Pulso se contrastan carga de mercado, selección, histórico, favoritos y errores con sus FR, y se comprueba que los números procedan de respuestas reales o capturas identificadas.

La lectura del código debe concentrarse en las decisiones con efecto visible: codificación de IDs y consultas, cálculo porcentual, limpieza de muestras, secuencias contra respuestas tardías y manejo de localStorage bloqueado. Las cadenas externas se escapan al formar HTML. Esto reduce un riesgo concreto, aunque no equivale a una auditoría completa de seguridad.

Criterio de aceptación humana: al menos un integrante debe poder explicar los módulos y reproducir un caso normal y uno de error. El resto del equipo debe revisar las decisiones que afectan requisitos, sin aceptar el resultado únicamente por su apariencia o por haber sido generado automáticamente.

Revisión de la validación

Pregunta de revisión: ¿las pruebas demuestran los criterios y sus límites son aceptables? Validate combina verificación automatizada y juicio humano. Si el código incumple un requisito válido se corrige; si el requisito está equivocado se revisa explícitamente la especificación [1, p. 4].

La validación técnica de esta entrega confirma 20 pruebas aprobadas y sintaxis correcta. La revisión visual previa aporta evidencia de interacción y adaptación móvil. Quedan por fortalecer pruebas automáticas de interfaz, orden de respuestas de red, búsqueda remota completa, precisión de activos muy pequeños y accesibilidad. El equipo debe decidir si estos límites son aceptables para la demostración académica.

Acta de cierre para el equipo

El estado técnico se presenta para revisión. La columna final se completa únicamente cuando el grupo realice su deliberación; no se asignan nombres ni se simulan aprobaciones.

Etapa	Base para decidir	Decisión del equipo
Requisitos	Alcance cripto, FR y escenarios.	Pendiente de registro.
Diseño	Arquitectura y dependencia de API.	Pendiente de registro.
Implementación	Código, trazabilidad y correcciones.	Pendiente de registro.
Validación	20 pruebas, evidencia visual y límites.	Pendiente de registro.

Registro sugerido del acta: fecha de revisión, cuatro participantes, observaciones, requisitos afectados, decisión y acciones pendientes. La revisión posterior a la construcción no debe presentarse como una aprobación previa a cada fase.

6 Discusión del código generado

Aportes de la generación asistida

La IA produjo la estructura HTML, estilos, módulos de lógica e interfaz, pruebas y documentación. Spec Kit aportó infraestructura y guías; no fue un generador determinista del producto. La especificación redujo decisiones implícitas al fijar moneda, rangos, estados y responsabilidades. No contamos con un experimento comparativo que permita afirmar un porcentaje de ahorro de tiempo o de errores.

Cálculos que pueden revisarse por separado

La función stats de core.js calcula los extremos y el rendimiento sobre muestras válidas. El siguiente fragmento corresponde al cálculo implementado; values contiene los precios normalizados:

```
return {
  min: Math.min(...values),
  max: Math.max(...values),
  change: values[0]
    ? (values.at(-1) / values[0] - 1) * 100
    : null,
};
```

Probamos la secuencia 100, 80 y 120: devuelve mínimo 80, máximo 120 y rendimiento cercano al 20 %. La ligera diferencia decimal se debe a la representación binaria y la prueba utiliza tolerancia. Si el primer valor es cero, el rendimiento queda ausente. Los extremos describen muestras; no se presentan como máximos y mínimos OHLC de cada intervalo.

Control de respuestas tardías

En loadHistory de app.js cada petición recibe una secuencia. Tras esperar a CoinGecko se compara con la selección vigente:

```
const seq = ++state.historySeq;
// La función captura también el activo y los días.
const r = await api.history(id, days, force);
if (seq !== state.historySeq) return;
```

El fragmento abreviado conserva el mecanismo real y añade un comentario explicativo. La misma comprobación se usa en rutas de error y finalización. Es una protección útil ante cambios rápidos, pero la evidencia actual combina inspección y prueba manual; no existe todavía una prueba de interfaz que fuerce todos los órdenes de respuesta.

Mantenibilidad y datos externos

core.js permite inyectar fetch y reloj para probar fallos y TTL sin depender del proveedor. En cambio, app.js concentra eventos, estado, HTML y dibujo SVG. Para una evolución mayor convendría separar tabla y gráfico, manteniendo el comportamiento. La validación del JSON comprueba estructura básica; no demuestra la integridad completa de cada campo financiero. Los tests existentes deben crecer con ese contrato.

7 Proceso y resultados de validación técnica

Se repitieron la comprobación de sintaxis de app.js y core.js y las 20 pruebas de tests/core.test.mjs sobre el código entregado. Resultado: **20 aprobadas, 0 fallidas, 0 omitidas**. Esta ejecución no modificó el producto. Los casos HTTP usan dobles de prueba; no prueban disponibilidad permanente de CoinGecko.

Grupo de pruebas	Casos	Qué verifican
Rangos y cálculo	5	Rangos permitidos, limpieza, estadísticas, vacío/base cero y rechazo de rango inválido.
Búsqueda y formato	3	Búsqueda local, valores ausentes y codificación de la consulta.
Favoritos	2	Persistencia por ID y recuperación ante almacenamiento corrupto o bloqueado.
Caché y concurrencia	3	TTL, actualización manual y deduplicación de consultas idénticas.
Fallos y contrato HTTP	5	429, 403/500, mercado inválido, histórico inválido y timeout.
URLs de histórico	1	Cuatro temporalidades generan parámetros distintos.
Captura de respaldo	1	Origen declarado, 50 activos y cuatro duraciones históricas diferentes.

Qué aportan las comprobaciones adicionales

Los mocks hacen reproducibles errores difíciles de provocar: timeout, restricción y límite de consultas. Las descargas y la navegación reales complementan esa evidencia. La prueba de snapshot comprueba estructura y contenido guardado; por sí sola no autentica criptográficamente su origen. Su procedencia se sustenta en el script de captura y el registro de la construcción.

Hallazgo adicional de precisión

Se ejecutó money(0.00000001) y el formato actual devolvió **$0.0000**. La función limita a seis decimales los precios inferiores a un dólar. Así, una cantidad positiva muy pequeña puede parecer cero. Este caso no está cubierto por las 20 pruebas y se registra como mejora pendiente de presentación financiera: adaptar la precisión a la magnitud o usar notación científica y añadir una regresión.

Límites de la conclusión

Las pruebas aprobadas respaldan los comportamientos cubiertos; no equivalen a cobertura total, verificación formal ni certificación para producción. No hay pipeline CI, medición de cobertura, auditoría completa WCAG o suite automatizada de aceptación del navegador. El dictamen técnico es aptitud para demostración académica con las restricciones documentadas y con aceptación humana final pendiente.

8 Evidencia de interfaz y trazabilidad

La captura muestra el rastreador publicado durante la construcción. Los precios son un estado histórico de la consulta, no cotizaciones válidas para cualquier fecha. La evidencia móvil y el registro de interacciones se conservan en docs/evidence/.

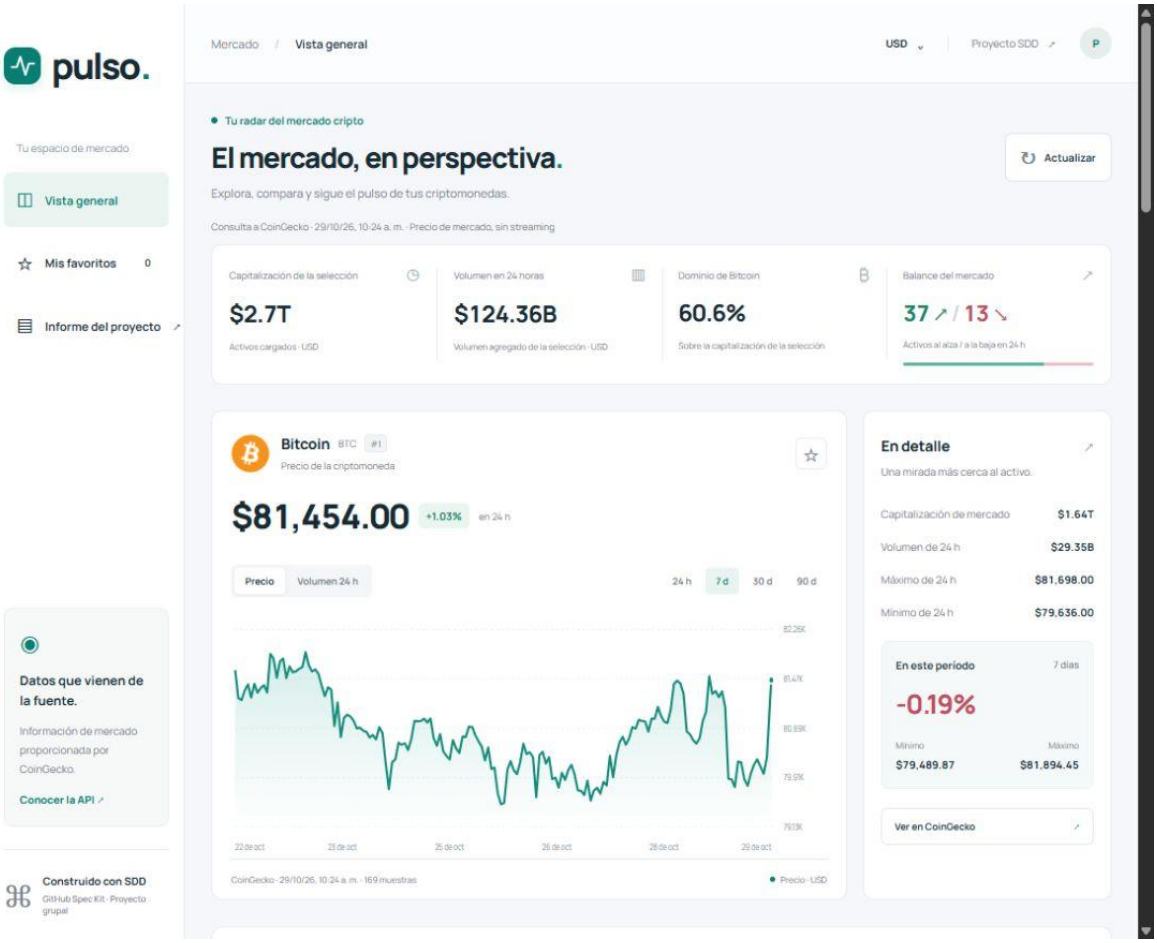


Figura 1 Página de Pulso publicada el 29 de octubre de 2026. Fuente: captura del proyecto.

Requisitos	Archivo y evidencia principal
FR 01 a FR 04	Tabla y búsqueda en app.js; filtros y formato en core.js; selección y AAPL comprobados en navegador.
FR 05 a FR 08	Históricos y SVG; rangos, estadísticas, contrato y secuencias revisados.
FR 09 a FR 11	Favoritos, orden y caché; pruebas de persistencia, TTL, concurrencia y errores.
FR 12 y FR 13	Artefactos SDD e informe; capturas de escritorio/móvil y navegación por teclado.

9 Discusión del proceso y mejoras prioritarias

Lo que el flujo permitió comprobar

La especificación hizo visibles decisiones que un prompt breve habría dejado abiertas: qué proveedor usar, cómo identificar cada activo y cómo representar rangos y fallos. El plan separó cálculos de interacción y permitió probar errores sin red real. La trazabilidad relacionó requisitos, archivos y evidencia, de modo que una corrección puede evaluarse por su efecto sobre el contrato acordado.

La validación produjo cambios concretos: se corrigió el desbordamiento móvil y se revisó la recuperación de favoritos. El límite real del proveedor confirmó la utilidad del estado de captura. Aun así, la misma herramienta que ayudó a generar código participó en su revisión, por lo que la aceptación por integrantes del grupo sigue siendo una comprobación distinta y necesaria.

Desviaciones y decisiones pendientes

La principal desviación frente al ciclo ideal fue formalizar el scaffold y la constitución después de comenzar la implementación. Además, los registros no acreditan revisión humana entre etapas ni ejecución independiente de todas las skills de Spec Kit. La existencia de plantillas y tareas no basta para afirmar que cada puerta de revisión se cerró. La práctica debe valorar la evidencia efectiva, no solo la cantidad de documentos.

Prioridad	Mejora	Motivo de validación
Alta	Registrar la revisión final de los cuatro integrantes.	Confirmar que requisitos, alcance y límites son aceptables.
Alta	Ajustar precisión de precios pequeños y agregar test.	Evitar mostrar como cero un precio positivo.
Alta	Automatizar aceptación y respuestas de red desordenadas.	Fortalecer evidencia sobre interfaz y FR-07.
Media	Ampliar contrato de respuestas y casos de búsqueda remota.	Detectar campos numéricos o resultados externos inválidos.
Media	Añadir CI y revisar accesibilidad con herramientas y usuarios.	Mantener la alineación y ampliar la verificación.

Conclusión del equipo

Pulso materializa el núcleo solicitado para criptomonedas y aporta gráficos, seguimiento y manejo explícito de datos. Los resultados de prueba y las evidencias visuales respaldan su uso como caso académico de implementación asistida por especificaciones. El siguiente paso del grupo es revisar los hallazgos, registrar su aceptación y priorizar las mejoras; una aprobación informada debe incluir las limitaciones de precisión y dependencia de CoinGecko.

10 Reproducción y referencias

Cómo repetir la validación

Desde la carpeta pulso y con Node.js 20 o superior se puede comprobar el mismo conjunto de pruebas y abrir el sitio localmente. Los comandos no requieren instalar dependencias de producción:

```
npm run check
npm start
# Abrir http://127.0.0.1:4173
```

Para revisar la aceptación, buscar BTC y ETH, seleccionar Ethereum, alternar los cuatro rangos, cambiar a volumen, recorrer el tooltip con flechas, marcar un favorito y recargar. Buscar AAPL debe mostrar el alcance de criptomonedas. Ante un límite del proveedor, comprobar la fecha y la etiqueta de captura o el error explícito. npm run snapshot descarga un nuevo respaldo y necesita acceso a CoinGecko; no conviene ejecutarlo repetidamente durante la exposición.

Cómo repetir el ciclo de Spec Kit

La integración instalada utiliza skills de Codex. El equipo debe ejecutar cada etapa de forma separada, revisar su resultado y registrar la decisión antes de continuar. Los nombres siguientes son instrucciones para el agente, no comandos de PowerShell:

```
$speckit-constitution
$speckit-specify
$speckit-plan
$speckit-tasks
$speckit-implement
$speckit-converge
```

Las instrucciones y prompts de reproducción no se presentan como una transcripción de seis ejecuciones previas. Los comandos realmente utilizados y sus resultados se encuentran en docs/INFORME.md y docs/evidence/spec-kit-init.txt. La revisión final de conformidad debe conservar los mismos requisitos como fuente de intención.

Referencias

[1] Piskala, D. B. (2026). Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants. Technical Report, arXiv:2602.00180v1. https://doi.org/10.48550/arXiv.2602.00180. PDF proporcionado para esta actividad. Se utilizaron pp. 1-3 para el espectro de SDD, pp. 3-4 para fases y revisión humana, p. 5 para Kiro y Spec Kit, y p. 7 para riesgos y disciplina de mantenimiento.

[2] GitHub. Spec Kit. Repositorio y documentación oficial consultados durante la implementación. https://github.com/github/spec-kit y https://github.github.io/spec-kit/installation.html. Versión instalada en el proyecto: Specify CLI 1.2.5.

[3] CoinGecko. API y documentación de mercados e históricos. https://www.coingecko.com/en/api y https://docs.coingecko.com/demo/reference/coins-id-market-chart. Fuente del contrato y de las consultas documentadas en la construcción.

[4] Proyecto Pulso (2026). Especificación, plan, código, tests y evidencias locales. Archivos principales: specs/001-crypto-tracker/spec.md, dist/core.js, dist/app.js, tests/core.test.mjs y docs/evidence/. Las conclusiones técnicas de este informe proceden de esos artefactos y de la repetición de pruebas.